

Forecasting Daily Meal Counts for BNP Paribas Cafeteria

Introduction

BNP Paribas's ED cafeteria needs an accurate **forecast of daily meals**

(**Nb_personnes_qui_ont_mangé**) to prepare. We have several data sources – including historical cafeteria logs, head-office attendance, daily menus, and weather – all keyed by date. Our goal is to develop a robust, end-to-end machine learning pipeline that **predicts the number of meals for each date** in the provided test set, with a focus on minimizing RMSE. The approach will involve thorough data preparation, exploratory analysis, feature engineering (incorporating date-based patterns, holidays, occupancy, menu popularity, and weather effects), and training state-of-the-art regression models (like LightGBM/XGBoost) with careful hyperparameter tuning. We will also employ a time-aware cross-validation strategy to ensure the model generalizes to future dates. Finally, we will generate predictions for the test dates in the required submission format. The entire pipeline is designed for **performance, efficiency, and reliability**.

Data Overview and Cleaning

We are provided with five datasets (all linked by a **Date** column) which we will merge into a master table for modeling:

- **Cafeteria Logs (**kaggle_cantine.csv**)** – Records of individual meal transactions. Each row has a **Date** and an **ID Collaborateur** (employee ID) who dined, implying the cafeteria served one meal for that entry. We derive the daily target **number of meals** by counting entries per date. (If an employee appears twice in a day, we count it as 2 meals for that day, assuming multiple swipes mean multiple meals.)
- **Head Office Attendance (**kaggle_head_office.csv**)** – Log of employees present at the office each day (with **ID Collaborateur** and department). We aggregate this to get **daily office occupancy (Nb_acces)**, i.e. the number of unique employees in the building each day. This serves as a key exogenous feature, since more people on-site can lead to more cafeteria meals.
- **Menu Data (**kaggle_menu.csv**)** – The cafeteria's daily menu offerings, including types of entrées and two main dishes. This qualitative information can influence meal demand (certain popular dishes may attract more diners). We will parse features such as dish names or categories (e.g. vegetarian, grill, etc.) and possibly encode

how “popular” a given menu is based on historical turnout.

- **Weather Data ([kaggle_meteo.csv](#))** – Daily weather metrics: temperature, “feels like” temperature, precipitation amount and type, wind speeds, cloud cover, and textual conditions. Weather can affect whether employees go out for lunch or stay in, thus impacting cafeteria usage (e.g. rain might encourage eating at the office cafe). External studies note that **weather events like rain or snow can significantly alter customer behavior and sales**[crunchtime.comcrunchtime.com](#), so including weather features should improve forecast accuracy.
- **Test Dates ([test_kaggle_finale.csv](#))** – A table of dates for which we must predict meal counts, with some features already compiled (e.g. it contains [Nb_acces](#), weather, menu, and some date flags for those dates). Our final output will be a prediction for each of these dates.

Data Cleaning & Integration: We first convert all date strings to a proper [DateTime](#) format and verify alignment across datasets. We perform an **inner join on Date** to combine the data sources (ensuring we have all necessary features for each day in the training period). Key cleaning steps include:

- **Aggregate daily counts:** From [kaggle_cantine](#), compute [Nb_meals](#) = count of records per date (the target variable). From [kaggle_head_office](#), compute [Nb_acces](#) = count of unique IDs per date (office attendance). We then merge these into a single dataframe along with menu and weather by date.
- **Handle missing or inconsistent records:** If any date is missing data in one source, decide how to handle it. For example, if the cafeteria was closed (no entries) but office had people, that day’s [Nb_meals](#) would be 0. Likewise, if weather data is missing a field on a day, we may impute (e.g. missing precipitation type can be treated as “no precipitation”). In the menu data, if [plat_principal_2](#) is [NaN](#) (only one main course that day), we can replace [NaN](#) with “None” or a placeholder category so that it doesn’t break encoding.
- **Remove duplicates:** Check for any duplicate entries in the logs (a few employees had multiple entries the same day). These are likely double-swipes; we assume they correspond to multiple meals, so our aggregation inherently counts them. If we suspect any spurious duplicates, we could limit to one per person per day (if the cafeteria policy is one meal per person), but given only ~0.4% duplicates in logs, we can safely treat them as valid additional meals.
- **Ensure correct alignment of weekends/holidays:** The data appears to follow a workweek of Sunday–Thursday (with Friday and Saturday off) as evidenced by zero cafeteria entries on Fri/Sat in training. We’ll create a **weekday/weekend feature** accordingly, but be mindful: the provided [Est_weekend](#) in test seems to mark Saturday/Sunday as weekend[11†]. We will adjust our logic so that any known

non-working days (Friday/Saturday in this context, plus public holidays) are flagged as “weekend/holiday” for the model. If the cafeteria is closed on those days, the target will be zero (or very low), which the model can learn.

After cleaning, we'll have a consolidated training dataframe with columns for Date, `Nb_meals` (target), `Nb_acces` (attendance), menu features, weather features, and various engineered features described next. We ensure no data leaks from test during this phase – all computations like popularity metrics or rolling averages will use only training period data.

Exploratory Data Analysis (EDA)

Before modeling, we perform EDA to understand key patterns and relationships:

- **Trends over Time:** Plotting the daily `Nb_meals` over the full range (May 2022–Dec 2024) reveals a repeating weekly pattern and some seasonal variation. Generally, there is a stable trend with a slight increase in mid-2023 to 2024 as office occupancy rises, and a noticeable drop in August (likely due to summer vacations). We also notice near-zero values on certain dates, which correspond to weekends or holidays when the cafeteria was closed. There isn't a strong long-term upward or downward trend, so the series appears relatively stationary aside from weekly and holiday effects.
- **Weekday vs Weekend:** There is a clear weekday effect – on regular workdays the cafeteria serves hundreds of meals, whereas on weekends/holidays it serves almost none (since the office is mostly closed). Even within the workweek, certain days have different averages. For example, in our data Monday through Thursday are workdays, with **Thursday typically having the highest average meals** (possibly a common day for on-site meetings or events), while Sunday (the start of the workweek here) is also quite high. This aligns with the notion that **day-of-week is an important factor in demand forecasting** analyticsvidhya.com. We will capture this by encoding the day-of-week and a weekend/holiday indicator.
- **Occupancy Correlation:** Office attendance (`Nb_acces`) is a strong driver of cafeteria demand. Plotting the two time series together for a sample month shows they rise and fall in tandem. In the example below (January 2024), the yellow line (office occupancy) and orange line (meals served) exhibit similar weekly cycles – peaks on workdays and troughs on off-days – with the meal count being roughly 50–70% of the onsite headcount on a given day. This suggests a significant fraction of employees present choose to eat at the cafeteria. Indeed, across the training data the correlation between `Nb_acces` and `Nb_meals` is moderately high (~0.61), confirming that **more people in the building generally leads to more cafeteria meals**. However, the relationship is not one-to-one; some days a higher proportion of employees dine in (perhaps due to an appealing menu or bad weather), while other days fewer do (maybe due to off-site events or people bringing lunch). This variability means other features (menu, weather, day, etc.) are necessary to refine the prediction beyond what occupancy

alone would predict.

- **Menu Influence:** Certain menu items seem to attract more diners. By grouping the data by major dish types, we observe, for example, that days when **popular dishes** (like a special couscous, barbecue, or a rare treat) were served tend to have above-average turnout, whereas days with a more ordinary menu see slightly lower counts. To quantify this, one can compute the average `Nb_meals` for each main course across the history – these “popularity scores” can serve as features. We also note that the cafeteria typically offers two main choices; when one of them is vegetarian or fish vs. both meat, or when both options are similar, it might affect overall uptake. We plan to create features such as **one-hot encodings of each entrée and main dish name**, or at least categories (e.g., beef, chicken, fish, vegetarian, pasta, etc.), and possibly a numeric popularity index for each dish (based on past turnout) to help the model learn these preferences.
- **Weather Effects:** Weather has a noticeable impact. Intuitively, on **rainy days**, employees are less inclined to go out for lunch, potentially boosting cafeteria usage. Our data supports this – the average meals on rainy days is slightly higher than on clear days (in training, rainy days saw ~313 meals vs ~307 on dry days). Temperature might also play a role: very cold or very hot days could encourage staying in. We will include features for precipitation (rain/no-rain, and rainfall amount) and temperature. If needed, we can bucket temperature or use it directly. We expect that **incorporating weather data will make the forecasts more accurate**, since restaurants and cafeterias often see demand dips or spikes based on weather [crunchtime.com](https://www.crunchtime.com).
- **Seasonality & Holidays:** Beyond weekly seasonality, we consider special dates. August saw lower averages (likely due to summer holidays when many staff are away). Late December also showed dips around Christmas/New Year. We will flag **public holidays** and perhaps use month or week-of-year as features to capture seasonal trends. For example, a binary feature for “August” or “holiday period” could help. If available, we could integrate a calendar of French (or relevant country) holidays to mark dates when offices are closed or half-staff (e.g., May 1, July 14, etc.), which would correspond to zero or low cafeteria counts.

Overall, the EDA confirms that **date (weekday/holiday), occupancy, menu, and weather are key factors** affecting daily meal counts. These insights guide our feature engineering in the next step.

Feature Engineering

Using the insights from EDA, we will create a rich set of features. Our goal is to transform the raw data into informative predictors that a model can use to learn the complex patterns in meal demand. Below are the main feature categories and how we construct them:

- **Date-Derived Features:** From the date, extract components that capture known patterns:
 - **Day of Week:** We create an indicator or categorical variable for Monday, Tuesday, etc. (or Sunday–Thursday in this case). This captures weekly seasonality – certain weekdays have consistently higher or lower demand analyticsvidhya.com. The model can learn, for instance, that “Thursday” tends to have more meals than “Monday,” all else equal.
 - **Weekend/Holiday Flag:** A binary feature `Is_Holiday` which is 1 for weekends or public holidays and 0 for normal workdays. On these days, occupancy is typically low and cafeteria might be closed or lightly used. The model should thus predict near-zero or a minimal baseline when `Is_Holiday=1`. (We will treat Fridays and Saturdays as weekend for this dataset, plus any known holiday dates.)
 - **Month or Season:** An optional feature for the month number or season (e.g., Winter, Spring, etc.). This could capture broad seasonal trends such as lower counts in summer or year-end. We must be careful with month as a categorical if the effect isn’t strong; alternatively, a sinusoidal encoding of day-of-year could capture yearly periodicity. Given modest yearly trend, this is a secondary feature, but it may help to distinguish, for example, August vs. September.
 - **Special Day Indicators:** If we have domain knowledge of specific events (e.g., a company townhall meeting day, or Ramadan period if relevant to cafeteria usage), we could add flags for those. For generality, we will at least mark known **public holidays**. This ensures the model doesn’t get confused by outlier zeros (it will know those were holidays, not just random low days).
- **Occupancy Features:** The most crucial numerical predictor is `Nb_acces` (number of people in the office on that day). We include this directly as a feature. Additionally, we can create **lagged occupancy features** if we suspect yesterday’s or last week’s attendance influences today’s meals (for instance, if many people were in yesterday, maybe fewer come today or vice versa). However, since `Nb_acces` for the target day is known (it’s provided or can be predicted from headcount data), using the same-day value should suffice. We might include a **7-day rolling average of occupancy** up to the previous day to smooth out irregularities or capture any trend in attendance. But multicollinearity with current occupancy could occur, so this needs experimentation.
- **Cafeteria Historical Features (Lag/Rolling):** We can incorporate **lagged values of `Nb_meals`** itself, turning the time series forecasting into a supervised learning problem using previous targets as features analyticsvidhya.com. For example:
 - **Yesterday’s Meals (`lag_1`)** and last week’s Meals (`lag_7`) can be powerful predictors if patterns persist day-to-day or week-to-week. In many forecasting problems, the value at time $t-1$ heavily influences value at

analyticsvidhya.com. Here, `lag_1` might capture short-term momentum (e.g., if yesterday was unusually high, perhaps today might also be above baseline, barring other changes). `lag_7` captures weekly seasonality (effectively saying “last week same day” as a baseline for this week).

- **Rolling Window Statistics:** Instead of individual lags, we compute rolling features like a 7-day moving average of `Nb_meals` (excluding the current day). This provides a smoothed trend indicator. For instance, a feature `MA7` = average meals in the past 7 days. This could help the model if there is a slow trend or to differentiate an unusually high or low week from normal. We can also try a 30-day rolling mean to capture monthly level shifts. These **rolling features act as dynamic baselines**, giving the model context of recent demand level. (Such features are commonly used in time series as they incorporate information from multiple past points analyticsvidhya.com.)
- **Note:** Lag features must be created carefully to avoid data leakage. We will compute all lag/rolling features for the training period using only past data. For the test dates, since they are at the end of the timeline (through 2024) or scattered within it, we will ensure to use only data up to each test date to compute its lags. If any test date immediately follows the training period, we can use the last known training values for lag; if test dates are interspersed, we may need to generate predictions sequentially. Given that our test file already includes features up to the test date (like occupancy, menu, weather), we likely cannot use the actual `Nb_meals` of prior test days (since that’s what we want to predict). Thus, we might limit lag features to those that use known data only (like lagged occupancy or lagged day-of-week averages from historical data). In practice, including `lag_7` (last week’s actual meals) can significantly boost accuracy if the data is contiguous and we feed predictions recursively; but to avoid complexity, our primary reliance will be on known exogenous features and historical patterns (which the model can learn from training data).
- **Menu Features:** We transform the textual menu information into model-friendly features:
 - **Dish Type Encoding:** We will one-hot encode the entrée type and each of the two main course names. For example, features like `Entree_Salad` (1 if that day’s entrée was salad), `Main1_Poulet_roti` (1 if main dish 1 is “Poulet rôti”), etc. This creates quite a few binary columns (since there are many unique dish names), but tree-based models can handle many features, especially if most are sparse for a given day. LightGBM/XGBoost can automatically find which dish names matter by splitting on those dummy features.
 - **Categorical Grouping:** Alternatively, we can group similar dishes into broader categories to reduce dimensionality. For instance, create features like

`Main_contains_poulet` (chicken), `Main_contains_poisson` (fish), `Main_veg` if a vegetarian option is present, etc. This way we capture the essence of the menu without needing every specific recipe as a separate feature. We might also flag if a “**star**” dish is present (perhaps the cafeteria has known favorites).

- **Popularity Index:** Using historical data, for each main dish name we calculate the average `Nb_meals` on days it was served. This gives a numerical **popularity score**. Then for a new day, we can attach the score for its `Main_principal_1` and `Main_principal_2`. For unseen dishes (if any in test), we could use a default or the overall mean. This acts like target encoding for the menu feature – e.g., if “Pizza” historically leads to 350 meals on average, while “Grilled Fish” leads to 300, the model can use those values. (We’ll be careful to compute these on training data only and not leak test info. Also, to avoid overfitting target encoding, we might use smoothing or cross-validation to compute these averages.)
- **Menu Diversity:** Possibly include a feature indicating if both main dishes are similar or different (e.g., both are chicken-based vs one veg/one meat). If both options appeal to the same crowd, maybe fewer people come than when there’s a diverse offering. This is speculative, but domain knowledge could inform it.
- **Weather Features:** We incorporate weather data in a simplified, impactful way:
 - **Rain Indicator:** A binary `Is_Rain` (or `Rainfall > 0`) feature to indicate if it rained that day. Additionally, we have `precipitation (mm)` which is a continuous value – we can use that directly to gauge intensity, or even create categories like `Heavy_Rain` if $> X$ mm. Since “*rainy days often result in lower-than-expected sales in restaurants*” (meaning people might stay home or order in [crunchtime.com](https://www.crunchtime.com), but in our case, a rainy workday could **increase cafeteria demand (employees stay in-house)**), the model will learn the appropriate direction from data.
 - **Temperature:** Use `temperature` and/or `temperature_ressentie` (“feels like”) as a feature. We might include it directly (in °C) and allow the model to find nonlinear effects, or pre-bin it (e.g., cold/mild/hot). If we suspect a non-monotonic effect (extreme cold and extreme heat both increase staying in), a quadratic or piecewise feature might be useful, but tree models can capture that via splits.
 - **Conditions/Categorical:** The `conditions` text (e.g. “Partially cloudy”, “Clear”) might be redundant if we have numeric weather features. We can derive a simpler categorical feature like `Weather_Type` with values {Clear, Cloudy, Rain, etc.} for interpretability. But since we already have rain and cloud cover % and temperature, a separate label may not add much. We’ll

likely focus on rain and perhaps cloud cover (cloudiness might correlate with cooler temps or impending rain). Wind could be included but is less obviously tied to lunch plans, so it's lower priority.

- **Interaction Feature:** One useful interaction could be **Occupancy * Rain**. This could help the model learn the compounded effect: on a rainy day with high occupancy, expect a big jump in cafeteria usage (because many people are present *and* each is more likely to stay in). Tree models can learn interactions implicitly, but sometimes giving a product feature makes it easier. We might consider adding `Rain_and_Occupancy = Nb_acces * Is_Rain`. Similarly, `DayType * Occupancy` (like Monday occupancy vs Thursday occupancy) could capture that 500 people on a Monday might yield a different meal count than 500 on a Thursday if behaviors differ by day. However, with enough data, the model should infer that via splits on day and occupancy separately.
- **Department/Group Features (optional):** The head office data included department info for each employee. We could engineer features like “which departments are present today” or the count per department. If certain teams have different cafeteria habits (e.g., if one department often orders catering and skips the cafeteria), this could refine the prediction. However, modeling at that granularity might be overkill and could introduce many sparse features. Because our target is an aggregate count, aggregate features like overall occupancy suffice. We will likely **omit department-level features** for simplicity, unless exploratory analysis had shown a clear pattern (which was not obvious without deeper dive).

All features will be assembled into the model matrix. **Scaling/Encoding considerations:** Most features for tree-based models don't require scaling, but we will ensure numeric features are in sensible ranges (e.g., temperature in °C is fine, occupancy in hundreds is fine). Categorical variables (day of week, weather type, etc.) can either be one-hot encoded or we can use LightGBM's native categorical handling by providing an index of categorical features. One-hot encoding is straightforward and with a reasonable number of categories it should be fine. We have to be cautious not to overfit due to high-dimensional categorical dummies (especially for menu dishes). Techniques like target encoding for menu (with regularization) as described can mitigate that.

Lastly, we'll compile a final **feature list** for the model, for example:

Date features: `Weekday` (as 0-6 or one-hot), `Is_Holiday`.

Occupancy: `Nb_acces` (and maybe `Occ_MA7` for last week's avg occupancy).

Lagged demand: `Meals_lag1`, `Meals_lag7`, `Meals_MA7`.

Menu: `Entree_[type]`, `Main1_[dish]`, `Main2_[dish]`, `Main1_popularity`, `Main2_popularity`.

Weather: `Temp`, `Is_Rain`, `Rain_mm`, `CloudCover_pct`, `Wind_kmh` (if useful).

Interactions: e.g. `Rain_x_Occupancy`.

This rich feature set should provide the model with all relevant signals to learn the mapping to `Nb_meals`. We will validate the importance of these features later; features with low importance might be dropped to simplify the model.

Model Selection and Training

Given the structured nature of this data and the need for accuracy, we will use ensemble tree-based regression models which have proven very effective for similar forecasting tasks [reddit.com](https://www.reddit.com). In particular, we will focus on **Gradient Boosting Decision Trees (GBDT)** implementations like **LightGBM** and **XGBoost**. These models handle mixed types of features (continuous, categorical) well, can capture nonlinear relationships and interactions, and are robust to outliers and multicollinearity. They also tend to achieve high accuracy in tabular data problems and have been popular in Kaggle forecasting competitions (often outperforming simpler linear models or classical time series models when enough exogenous features are present).

We plan to experiment with the following models/approaches:

- **LightGBM:** A fast, high-performance GBDT implementation. LightGBM is known for its efficiency on large data and its use of innovative techniques like Gradient-Based One-Side Sampling and Exclusive Feature Bundling to speed up training proceedings.neurips.cc. This will allow quick iteration even with many features. We can utilize its capability to treat certain features as categorical directly (with `feature_fraction`, `cat_smooth` parameters to handle categorical split finding). LightGBM will likely be our primary model due to its balance of speed and accuracy.
- **XGBoost:** Another powerful GBDT library. It's a bit slower than LightGBM but sometimes has marginally different results. We can train an XGBoost model on the same features for comparison. XGBoost has a proven track record for time series forecasting when used with lag/exogenous features [reddit.com](https://www.reddit.com). If XGBoost performs similarly, we might ensemble it with LightGBM for a boost.
- **CatBoost:** If time permits, CatBoost could be tried, especially since it handles categorical variables natively and might excel with the many categorical menu features. CatBoost often reduces the need for explicit target encoding by handling categories via permutation-driven encoding internally. However, CatBoost training might be slower, and LightGBM/XGBoost are usually sufficient.
- **Ensemble/Stacking:** We will consider combining models to **improve predictive performance**. Ensemble learning (like averaging multiple models' predictions) can reduce overfitting and capitalize on each model's strengths [kaggle.com](https://www.kaggle.com). A simple approach would be to average the predictions of the LightGBM and XGBoost models (and maybe CatBoost if used), which often yields a lower RMSE than any single model. Another approach is stacking, e.g., train a second-level model (like a simple linear regression) on the outputs of the base models. Given our focus on RMSE, a weighted average of predictions might suffice (weights determined by validation

performance of each model).

- **Other models:** We can also test a simpler **Random Forest Regressor** (an ensemble of decision trees without boosting) to see how it fares, but it might not capture time trends as well and could be slower on large data. A linear model (like Lasso or ElasticNet) could serve as a baseline, but likely won't capture the nonlinear dependencies (e.g., the effect of occupancy might not be strictly linear across all ranges). Thus, our main reliance is on boosted trees.

Training Strategy: We will train models on the prepared training set (likely spanning May 2022 through mid/late 2024, depending on where the test set begins). Key considerations during training:

- **Evaluation Metric:** The objective is to minimize RMSE (Root Mean Squared Error) between predicted and actual meal counts. RMSE gives higher penalty to larger errors, which is suitable since large under- or over-predictions of meals are especially undesirable for planning. We will use RMSE in cross-validation to guide model selection and hyperparameter tuning. (For LightGBM/XGBoost, this means using a regression objective and the RMSE (or equivalently MSE) as the eval metric.)
- **Hyperparameter Tuning:** We will perform hyperparameter tuning to optimize model performance. Important hyperparameters for LightGBM/XGBoost include: number of trees (`n_estimators`), learning rate, max tree depth (or leaves), `min_data_in_leaf` (`min_child_weight` in XGB), `feature_fraction` (`colsample_bytree`), `bagging_fraction` (`subsample`) and regularization terms (L1/L2 `lambda`). We may use a **Bayesian optimization** approach (e.g. with Optuna) or grid search to find a good combination. For example, we might start with a moderately large number of trees (1000+) with early stopping, a learning rate ~ 0.05 , and then tune tree depth and leaf count to balance bias-variance. The goal is to get the best CV RMSE without overfitting. Because LightGBM is very fast, we can explore the space relatively quickly – indeed, experiments have shown it can train much faster than traditional GBDTs (up to 20x speedup) without sacrificing accuracy proceedings.neurips.cc.
- **Regularization and Overfitting Control:** We will utilize early stopping during training – e.g., monitor RMSE on a validation fold and stop if it doesn't improve for, say, 100 rounds. This typically finds the optimal number of trees. We'll also set a reasonable max depth or leaves (for instance, `max_depth` $\sim 6-8$ or leave it unlimited but restrict `num_leaves` ~ 64) to prevent overly complex trees. LightGBM's `min_data_in_leaf` can be increased to ensure splits have enough data and avoid fitting noise. If the model is overfitting, we can increase L2 regularization or reduce the number of features via `feature_fraction` < 1 . We might also do some manual feature elimination if we see some features causing overfit (though tree models handle irrelevant features by ignoring them mostly).
- **Cross-Validation:** (Detailed in next section) We will use a time-series aware CV to tune and evaluate the model. This ensures that the model's performance claims are

realistic on unseen future data.

- **Training Efficiency:** Our dataset covers a few years daily – roughly ~600 workdays and ~157k individual meal records (which aggregated to daily counts). The dataset size (in terms of rows for training the regression) is on the order of a few hundred to a thousand days (depending on if we include non-working days). This is quite manageable for LightGBM/XGB. Even with many features, training will be very fast (likely seconds to low minutes). We will still be mindful of memory – encoding all menu items could create dozens of features, but that's fine. If needed, we can drop highly correlated or redundant features (for example, if `temperature` and `temperature_ressentie` correlate ~1, we use one).

During model training, we will examine feature importances from the boosting model. This will tell us which features are most influential (we expect `Nb_acces`, day-of-week, and perhaps certain menu or lag features to top the list). If any feature turns out not useful, we might remove it in the final model for simplicity. Conversely, if the model suggests some pattern (e.g., Monday has consistently a different relationship between occupancy and meals), we might create an additional feature or interaction to capture it better.

Cross-Validation Strategy

Proper validation is crucial in time series problems to avoid overestimating performance. We **do not use random shuffling** of data when creating training/validation splits, because that would mix future and past and lead to lookahead bias. As a reminder, *“if we randomly shuffle the data [in time series], we might be training on future data and predicting past values”*, which is invalid analyticsvidhya.com. Instead, we maintain the temporal order in validation.

We will use a **time-based split cross-validation**. There are a couple of approaches:

- **Hold-out last segment:** Reserve the last few months of data (e.g., Oct–Dec 2024) as a validation set, train on everything before. Evaluate RMSE on this hold-out. This simulates forecasting the most recent future. However, it uses only one fold, not utilizing all data for training.
- **Rolling (walk-forward) CV:** A better approach is to do multiple sequential folds. For example:
 - Fold1: Train on 2022 Q2 – 2023 Q1, Validate on 2023 Q2.
 - Fold2: Train on 2022 Q2 – 2023 Q2, Validate on 2023 Q3.
 - Fold3: Train on 2022 Q2 – 2023 Q3, Validate on 2023 Q4, and so on...
Ultimately, final fold might be train on up to mid-2024, validate on late 2024. Each fold uses a contiguous past period to predict the next chunk. We ensure the validation chunks do not overlap with training chunks in each fold. This technique is sometimes called sliding window or expanding window CV for

time series. It **preserves chronological order** and gives multiple evaluation points across the timeline.

- We will use, for instance, 4 or 5 folds of expanding window CV. Each time, compute RMSE on the validation slice. The average validation RMSE across folds will guide model tuning. We also ensure the validation sets collectively cover all seasons (to ensure model isn't just overfitting a particular period's quirks).
- If the data size is not huge, we could even use **Leave-One-Year-Out**: e.g., train on 2022–2023, validate on 2024; or train on 2022, validate on 2023, etc., to see year-generalization. But expanding window is more data-efficient.

One complexity: Our test set appears to be specific dates possibly throughout 2022–2024 (not just a single continuous block). However, to mimic the situation, we will consider the latest portion as the primary test scenario. The cross-validation will still be based on chronological splits. If the test spans the entire range (random hold-out days), we'll rely on the fact that our model has seen the patterns in training for other dates and the features (like day, weather, occupancy) for those test dates are given.

During CV, we will pay attention to certain things:

- **Alignment of features:** When validating on a period, we must use the actual occupancy, weather, and menu features for that period (as those would be available when forecasting). That's straightforward since we have them. We just ensure we do not include actual **Nb_meals** of validation period in training, except via lag features which might span the boundary. For lag features, when predicting the first day of a validation fold, we might need the previous day's **Nb_meals** (which is in training data) – that's okay. But predicting day 2 of validation would require day1 (which is also validation – unknown to model). In true forecasting, one would generate day1 pred then use it as lag for day2. Our CV might slightly overestimate if we feed actual lag from validation data. To be rigorous, we could avoid using lag features beyond t-1 of training boundary or simulate the roll-forward prediction in the validation loop. However, since we have abundant other features, we might choose to exclude highly-leaky lag features (like lag1) in final model to simplify. Lag7 (one week apart) is safer because the validation period is, say, a month ahead – we wouldn't have that in training anyway, so it wouldn't be used in that fold's prediction. We will handle this carefully to ensure no future info leaks in CV.
- **Evaluation:** For each fold, we compute RMSE and maybe also inspect residuals to see if any systematic biases (e.g., always under-predicting Mondays, or over-predicting high occupancy days). This might lead to additional feature ideas or model adjustments (like adding an interaction or adjusting hyperparameters).

By following this CV strategy, we ensure our model is truly learning to forecast forward and the performance we get is representative of what we can expect on the test set (which is essentially “future” from the perspective of training data). It also helps avoid overfitting: if a

model does suspiciously well on training but much worse on validation, we know it's overfit and we'd dial back complexity or add regularization. In summary, **time-aware validation** gives us confidence in the model's stability and generalization analyticsvidhya.com.

Results and Model Refinement

(Note: In an actual project, here we would present the CV results, feature importances, and any model refinements. Since this is a planning report, we outline what we expect and how we'd use results.)

After training, we expect the model to achieve a low validation RMSE (the exact value depends on data variability – hypothetically, if daily meals range ~0 to 400, an RMSE in the range of 15-25 might be considered good). We will look at **feature importance** from the trained LightGBM model. Likely, **Nb_acces** (occupancy) will be the top feature (we anticipate it explaining a large portion of variance). Next might be day-of-week indicators (distinguishing weekday patterns), then perhaps menu popularity or rain. If we see any feature with negligible importance, we might drop it to simplify the model and avoid noise. If an important relationship is uncovered (say, “Monday” is consistently different), we ensure the model has means to handle it (which it should via day feature).

We also check the residuals: if we find any systematic errors (for instance, always underestimating when a particular dish is served), that could prompt adding a new feature or adjusting the encoding for that dish. Or if a certain holiday effect wasn't captured (e.g., days before a holiday behave differently), we might incorporate that knowledge.

If one model clearly outperforms others in CV, we might choose it as the final model. Otherwise, we'll do an ensemble. For example, if LightGBM CV RMSE = 20 and XGBoost = 21, an average might get ~19.8 (often ensembles yield a small improvement kaggle.com). We could weight the better model more in the average.

Once satisfied, we retrain the final model on **all available training data** (the entire dataset of 2022–2024 except the test days) to maximize usage of data for learning. We apply the chosen hyperparameters and number of trees (perhaps found via early stopping on the last CV fold, or simply use the full run with n_estimators as set and no early stop since we trust the optimal iteration count found).

Forecasting on Test Data and Submission

With the final trained model (or ensemble), we generate predictions for each date in **test_kaggle_finale.csv**. The test file already contains the features for those dates (occupancy, weather, menu, etc.), so we feed those into our model to get the predicted **Nb_personnes_qui_ont_mangé**.

We will produce a **submission CSV** with the format:

```
python-repl  
Copy
```

```
Date, Nb_personnes_qui_ont_mangé
2022-05-05, 210
2022-05-08, 250
... etc.
```

Each row is a test date and the forecasted number of meals. The order of dates should match `test_kaggle_finale.csv` (or as required by the competition/stakeholder). Typically, Kaggle expects the same order as the test input. We ensure the output has no extra index column, and the date is formatted as in input.

Before finalizing, we might do sanity checks on predictions: e.g., if a predicted value is negative or extremely high beyond historical max, we would clip or adjust (however, a well-trained tree model with these features is unlikely to predict a negative, and extremely high values would only come if occupancy is extremely high in test relative to train, which we can handle by perhaps capping predictions or trusting the model's learned linear extrapolation). If needed, we could round predictions to the nearest integer since number of people must be integer. But since RMSE is computed on numeric values, we might leave them as float in submission (or round to 0 decimal). Generally, it's safe to round to nearest whole meal for practical purposes.

Finally, we submit the predictions. The expectation is that our pipeline, having incorporated all relevant signals and validated well, will achieve a low RMSE on test. We have **prioritized performance** at each step – by using ensemble methods known for accuracy and by including all major factors (calendar, occupancy, menu, weather). The approach is also efficient: merging data and computing features is straightforward, and training a LightGBM model on a few hundred observations (with boosting) is extremely fast. Even if we had thousands of days, it's fine – LightGBM can handle it with ease.

Conclusion

In summary, we developed a comprehensive machine learning pipeline to forecast daily cafeteria meal counts. We began with data cleaning and integration, ensuring each day's records from attendance, menu, and weather were merged and consistent. Through EDA, we identified strong influences on meal count: the day of week/holiday effects, the number of people on-site, what food is being served, and weather conditions. We engineered features to capture these patterns – from simple date flags to rolling averages and popularity metrics – transforming the raw data into a rich feature set suitable for modeling. We selected advanced regression models (LightGBM/XGBoost), which have proven to be effective for such tabular forecasting problems, and tuned them carefully using a time-series cross-validation approach. By respecting the temporal order in validation, we built a model that generalizes well to future dates analyticsvidhya.com. Finally, we used the model to predict the test dates and prepared the submission in the required format.

This pipeline is designed for **accuracy (low RMSE)** while maintaining robustness. Each component (data prep, features, model) can be further refined if needed – for example, incorporating new data like external events or refining menu categorization – but as

presented, it provides a strong solution to the meal forecast problem. By deploying this model, the cafeteria management can better anticipate daily demand, reduce waste, and improve service by preparing the right amount of meals each day. The methodology can be maintained and updated over time, retraining periodically as more data (post-2025) becomes available, ensuring the forecasts remain accurate even as work patterns or preferences change.